

Streaming large scale live events: How to transition to MOQ?

Nominoë Kervadec, Christoph Neumann, Guillaume Bichot
Broadpeak, France

Media over Quic (MOQ) is a new low-latency media delivery solution for ingest and distribution of media under standardization at the IETF. Eventually, MOQ should harmonize and simplify real-time (video) streaming infrastructures deployed for live and conversational applications. The MOQ Transport (MOQT) [11] protocol specifies a publish/subscribe control and a data delivery stack that: 1/is based on the QUIC protocol [6], possibly operating WebTransport [4], 2/defines a new container format complemented and constrained by LOC [14] and WARP [8] specifications that fits both conversational and adaptive streaming applications. In the case of live (video) streaming the ambition is to replace the current HTTP adaptive streaming (HAS) solutions (e.g., MPEG DASH or HLS). MOQ is based on the PUSH mode principle gathering publishers (media source), subscribers (media consumer) and relays in between playing the role of a switch (with cache capability). As soon as the client has subscribed to a video track, the publisher or the relay pushes the corresponding MOQT groups and objects continuously. While the PUSH approach is not fundamentally new and already exists within RTP/RTSP and the WebRTC ecosystem, there is a need for harmonization, conciliation, and standardization which is being tackled by the MOQ initiative.

It will take a couple of years before the entire streaming ecosystem (ingest, distribution, players) has completely transitioned to MOQ and that everyone will be able to enjoy all the benefits provided by MOQ. Nevertheless, the community started to test end-to-end a MOQT versus HAS protocols for instance [13] but none of these touched on the impact on existing infrastructures with a focus on the performance and architecture impact.

Transition path: rely on existing infrastructure

Our idea was to experiment MOQ support from the client perspective without requiring the full MOQ chain, i.e. without requiring MOQ publishers and full MOQ relays. The approach was to rely on an existing HTTP adaptive streaming CDN infrastructure (composed of origins and caches) and add in our cache server an HAS to MOQT relay (a half MOQT relay). It allows the support of MOQ clients/players that can benefit from MOQ without impacting the existing CDN infrastructure. In addition, our half MOQT relay would benefit from cache server ecosystem.

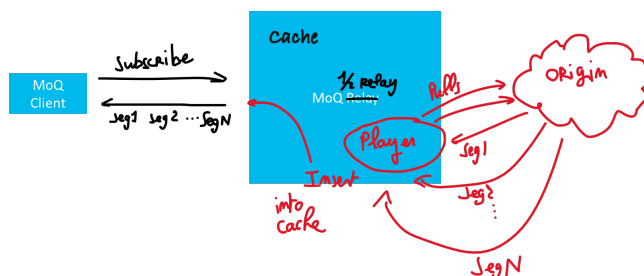


Figure 1: The 1/2 MOQ relay

Experiment

Our experimental platform gathers, server side, the Moxxygen MOQT relay [9], an HAS (DASH/CMAF) puller for fetching CMAF segments from the Origin server and an HTTP3 + Webtransport implementation from Meta [8] accompanied with various UDP optimizations including kernel bypass (DPDK). The MOQT framing has to be CMAF [10] friendly in order to facilitate the interconnection between MOQ and HAS domains; it relies on a simplified WARP catalog [8] while applying group and object packaging defined by CARP [7]. On the player side, we extended moq-js [12] to align MOQT versions and support CARP CMAF packaging. We have run concurrently the MOQ player and the dash.js player (in low latency mode with a live delay forced to 1s) over a MOQT and an HTTP2 session respectively, down-streaming the same live content from the same origin server (MPEG DASH compliant). Beyond validating the good behavior of our cache sharing the content among the two flows, We have run measurements campaign related to CPU consumption (server side) and quality of experience (client side).

Issues and WebTransport mitigation

Despite a very dynamic MOQ community, it will take time to transition from HAS to MOQT if ever happening. There are a couple of issues with the MOQ stack that we need to pay attention to and that influence what we call the transition path(s). First, according to our measurements campaign, streaming with MOQT appears to consume 3 times more CPU than with HTTPS. This is mostly due to the way TLS is embedded in QUIC; the QUIC framing depends on the MTU (Maximum Transfer Unit) and so is the overhead introduced by TLS. This is not the case with HTTPS (TLS framing is independent from the transport layer). This will certainly be mitigated in the future with hardware offloading but we are not yet there. Another issue is inherent to UDP. There are some contexts where simply UDP packets do not flow well or even do not flow at all. This is/was due mostly to ISPs protecting against cyber attacks (DNS, ...). Finally there is a third issue that may be related to the previous one that is the QUIC unfairness to TCP. This was clearly illustrated with our experiment. Having only one fixed track available on the MOQ side versus 3 DASH representations on the HAS/DASH side, during network overload the dash.js player always switch to the lower quality and starts to experiment freezes while the MOQ session remains stable. This has been shown by Choffnes et al. [3] and confirmed no matter the network of operation [2].

These issues can be mitigated with a way to switch back to TCP when operating QUIC makes the end user quality of experience at risk. One ongoing work that is promising is the support of WebTransport over HTTP and more specifically HTTP2 [1]. The WebTransport protocol has primarily been designed for HTTP3/QUIC as the successor of the WebSocket [5] designed for HTTP1.1/TCP. Because MOQT relies primarily on WebTransport, the backup to TCP operation would consist in switching the Webtransport pipe over HTTP2.

REFERENCES

- [1] T. Pauly M. Thomson G. Xie V. Vasiliev A. Frindell, E. Kinnear. 2025. *The WebTransport Protocol Framework*. Internet-Draft. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/draft-ietf-webtrans-http2-12> Work in Progress.
- [2] Panayotis G. Cottis Apostolos I. Kyrtatzis. 2022. QUIC vs TCP: A Performance Evaluation over LTE with NS-3. *Communications and Network* 14 (2022), 12–22.
- [3] David Choffnes Cristina Nita-Rotaru Arash Molavi Kakhki, Samuel Jero and Alan Mislove. 2019. Taking a Long Look at QUIC: An Approach for Rigorous Evaluation of Rapidly Evolving Transport Protocols. *Commun. ACM* 62, 7 (2019), 86–94.
- [4] V. Vasiliev E. Kinnear. 2025. *The WebTransport Protocol Framework*. Internet-Draft. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/draft-ietf-webtrans-overview-10> Work in Progress.
- [5] A. Melnikov I. Fette. 2011. *The WebSocket Protocol*. Request for comment RFC6455. IETF. <https://datatracker.ietf.org/doc/html/rfc6455>
- [6] Jana Iyengar and Martin Thomson. 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000. <https://doi.org/10.17487/RFC9000>
- [7] Will Law. 2025. *CARP - a CMAF compliant implementation of WARP*. Internet-Draft. <https://wilaw.github.io/carp/draft-law-moq-carp.html> Work in Progress.
- [8] Will Law, Luke Curley, Victor Vasiliev, Suhas Nandakumar, and Kirill Pugin. 2025. *WARP Streaming Format*. Internet-Draft. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/draft-ietf-moq-warp/01/> Work in Progress.
- [9] Meta. 2025. *Moxygen*. <https://github.com/facebookexperimental/moxygen>
- [10] MPEG. 2024. *Information technology — Multimedia application format (MPEG-A), Part 19: Common media application format (CMAF) for segmented media*. International Standard ISO/IEC 23000-19:2024. ISO/IEC. <https://www.iso.org/standard/85623.html>
- [11] Suhas Nandakumar, Victor Vasiliev, Ian Swett, and Alan Frindell. 2025. *Media over QUIC Transport*. Internet-Draft. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/draft-ietf-moq-transport/14/> Work in Progress.
- [12] video dev. 2025. *Media over QUIC (MoQ) Playe*. <https://github.com/video-dev/moq-js>
- [13] Deniz Ugur Yigit K. Erinc Ali C. Begen Zafer Gurel, Tugce Erkilic Civelek. 2024. Media-over-QUIC Transport vs. Low-Latency DASH: a Deathmatch Testbed. *MMSys '24: Proceedings of the 15th ACM Multimedia Systems Conference* (2024), 448–452.
- [14] Mo Zanaty, Suhas Nandakumar, and Peter Thatcher. 2025. *Low Overhead Media Container*. Internet-Draft. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/draft-ietf-moq-loc/01/> Work in Progress.